

Sistemas Distribuídos e Computação em Nuvem

Aula 9 — Tempo e ordenação: relógios lógicos

C2 · Coordenação e consistência · 01/10/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — C2

C1 · Fundamentos (1–7) ✓

C2 · Coordenação e consistência (8–12)

Mensageria ✓ · relógios lógicos · CAP · Raft ·
resiliência.

C3 · Nuvem e segurança (13–18)

📌 Aula 9. Aviso de largada 🖱️

💡 Este é, com sinceridade, o tópico **mais abstrato** do semestre. Por isso vamos **devagar**, por **exemplos numéricos** e uma **simulação** — **zero** fórmula decorada. Se não "clicar" de primeira, é normal: relemos a tabela e roda o código.

Retomada

 Você completou o worker (tentativa + dead-letter) e começou o C2.A2?

- Na fila, as tarefas eram processadas **em alguma ordem** — mas **qual** ordem é a "certa" quando as coisas acontecem em **máquinas diferentes**?
- Hoje descobrimos que **carimbo de hora não serve** para isso — e o que serve.

Objetivos desta aula

Ao final, você será capaz de:

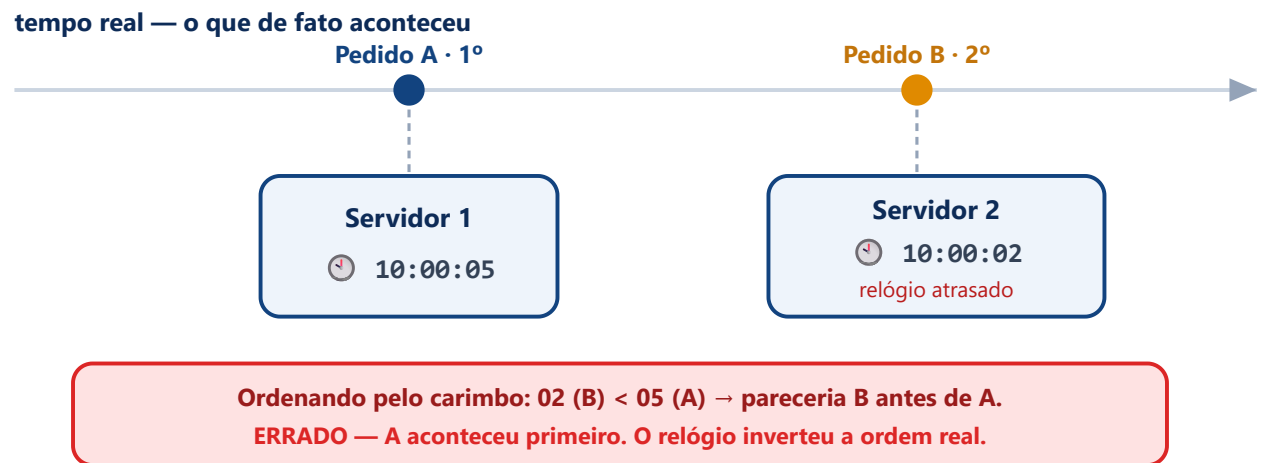
1. **Explicar** por que **não existe relógio global confiável** em sistemas distribuídos.
2. **Aplicar** o relógio de **Lamport** para ordenar eventos (a regra do **máximo + 1**).
3. **Usar** o relógio **vetorial** para identificar eventos **concorrentes**.

O problema: não existe um "agora" comum

- Cada máquina tem **seu próprio relógio**, e dois relógios **nunca** marcam o mesmo instante — eles andam em ritmos um pouco diferentes (**clock drift**, desvio de relógio).
- Sincronizar pela rede **não resolve de vez**: a própria mensagem de acerto leva um tempo **variável** para chegar.

⚠ Consequência séria: **não dá para confiar no carimbo de hora** para decidir a ordem dos eventos. Se dois pedidos chegam quase juntos a servidores diferentes, comparar os relógios físicos pode **inverter** a ordem real.

Por que o relógio engana — um exemplo



💡 **Em miúdos:** dois relógios de parede nunca batem igual. Se cada foto guarda a hora do *seu* relógio, **não dá para confiar nela** para dizer o que aconteceu primeiro.

A virada de chave

Se não podemos confiar na **hora**, o que sobra?

 Não precisamos saber **QUANDO** (a hora) — precisamos saber **O QUE VEIO ANTES DE QUÊ** (a ordem). Trocamos "hora real" por **causalidade**.

- **Causalidade**: a relação em que um evento **influenciou** a ocorrência de outro.
- É como **numerar suas cartas** para saber a sequência — mesmo sem olhar o relógio.

"Aconteceu antes" ($\rightarrow$) — 3 regrinhas

Dizemos que $A \rightarrow B$ ("A aconteceu antes de B") quando:

1. **Mesmo processo:** A e B estão na mesma máquina e A veio **primeiro**.
2. **Mensagem:** A é o **envio** e B é o **recebimento** da mesma mensagem (enviar vem antes de receber).
3. **Transitividade:** se $A \rightarrow B$ e $B \rightarrow C$, então $A \rightarrow C$.

⚠ Se **nenhuma** das regras liga A e B, eles são **concorrentes** ($A \parallel B$): aconteceram sem que um influenciasse o outro. Guardar esse termo — ele é a estrela da aula.

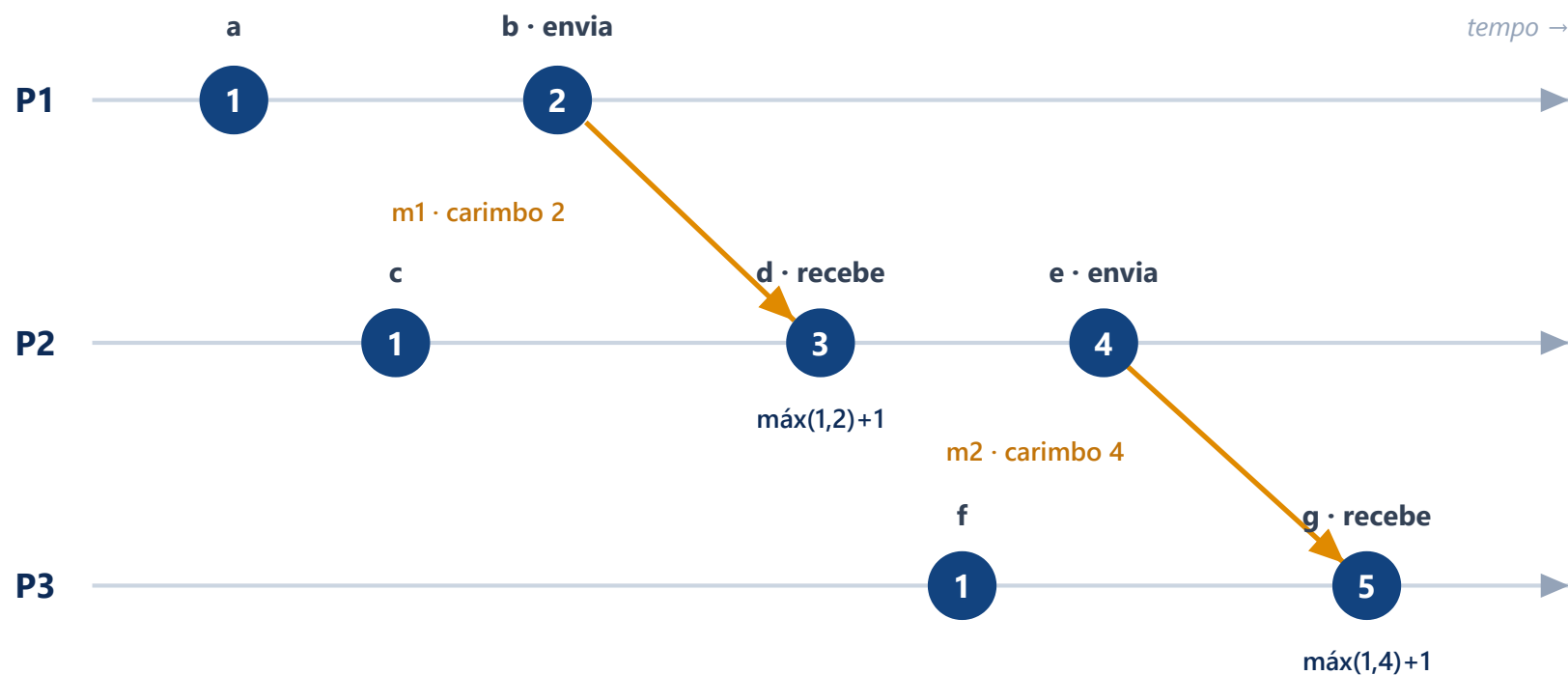
Relógio de Lamport — uma regra só

Um **contador por processo** (começa em 0). Três passos:

1. **Evento local**: contador **+1**.
2. **Ao enviar**: faça **+1** e **mande junto** o valor do contador (o "carimbo").
3. **Ao receber**: contador = **máximo**(o meu, o carimbo recebido) **+ 1**.

💡 É só isso: **máximo + 1** no recebimento; **+1** nos demais. Vamos ver rodando.

Exemplo — a execução (o número no círculo é o relógio)



💡 **Círculo azul** = um evento (o número dentro é o **relógio de Lamport**). **Seta laranja** = mensagem, que leva o **carimbo** junto. Nos recebimentos (**d**, **g**): **máximo + 1**.

Exemplo — o contador de Lamport passo a passo

evento	processo	o que acontece	cálculo	Lamport
a	P1	evento local	$0 + 1$	1
b	P1	envia m1 (carimbo 2)	$1 + 1$	2
c	P2	evento local	$0 + 1$	1
d	P2	recebe m1 (carimbo 2)	$\text{máx}(1, 2) + 1$	3
e	P2	envia m2 (carimbo 4)	$3 + 1$	4
f	P3	evento local	$0 + 1$	1
g	P3	recebe m2 (carimbo 4)	$\text{máx}(1, 4) + 1$	5

💡 Olhe a linha **d**: o carimbo (2) "puxou" o relógio de P2 para frente. É o **máximo + 1** em ação.

A garantia — e a pegadinha

- **Garantia:** se $A \rightarrow B$ (A causou B), então $\text{Lamport}(A) < \text{Lamport}(B)$. A causa **sempre** tem número menor que o efeito. 
- **Pegadinha:** a recíproca **NÃO vale**. Um número menor **não prova** que houve causa.

⚠️ Veja na tabela: **c** (P2, Lamport 1) e **a** (P1, Lamport 1) têm o **mesmo** número, mas são **concorrentes**. E **f** (1) < **b** (2) sem que um tenha causado o outro. Lamport **ordena**, mas **não distingue** concorrência.

Relógio vetorial — quem sabe o quê

Para **ter certeza** se dois eventos são causais ou **concorrentes**, cada processo guarda um **vetor**: um contador para cada processo.

- **Local/envio**: +1 na **sua** posição.
- **Recebimento**: pega o **máximo posição a posição** com o vetor recebido, depois +1 na sua posição.

💡 Em vez de "um número", cada evento carrega "o que **cada** processo já tinha visto". Isso é o suficiente para detectar concorrência.

Vetorial — os 3 casos (comparando posição a posição)

Com os vetores da nossa execução:

par	vetores	veredito
$d \rightarrow g$	$[2, 2, 0] \leq [2, 3, 2]$	causal (d causou g)
$c \parallel f$	$[0, 1, 0]$ vs $[0, 0, 1]$	concorrentes (nenhum $\leq$ o outro)

- Se um vetor é $\leq$ o outro em todas as posições $\rightarrow$ um **causou** o outro.
- Se **cada um é maior em alguma posição** (nenhum domina) $\rightarrow$ **concorrentes**.

 Detectar concorrência é a base para achar **conflitos entre réplicas** — quando duas cópias são escritas ao mesmo tempo (próxima aula, CAP).

Laboratório

Simular a ordenação — e achar o par concorrente

Lab · Passo 1 — a simulação (lamport_sim.py)

Descrevemos a **ordem real** dos eventos; o programa aplica as regras:

```
for proc, tipo, rotulo, msg in execucao:      # execucao = lista de eventos
    if tipo in ("local", "send"):
        lamport[proc] += 1                    # +1
        if tipo == "send":
            carimbo_l[msg] = lamport[proc]    # manda o relógio junto
    else: # recv
        lamport[proc] = max(lamport[proc], carimbo_l[msg]) + 1 # MÁXIMO + 1
    eventos.append((rotulo, proc, lamport[proc]))
```

💡 O arquivo completo (exemplos/aula09/lamport_sim.py) faz também o **vetorial** e lista os pares concorrentes.

Lab · Passo 2 — rodar e ler a saída

```
python lamport_sim.py
# evento proc  Lamport  vetor
# a      P1    1        [1, 0, 0]
# c      P2    1        [0, 1, 0]
# d      P2    3        [2, 2, 0]
# ...
# Pares concorrentes (nem um causou o outro):
#  c(P2) || f(P3)      <- dois eventos locais que nunca se falaram
```

💡 Experimente: **adicione um processo P4** e uma mensagem nova, rode de novo e veja quais pares viram concorrentes.

No seu trabalho — C2.A2

- No RAG, o **serviço de recuperação** compara a pergunta com os documentos por **similaridade** (conteúdo).
- A teoria de hoje reforça um cuidado: a recuperação deve depender do **conteúdo**, **não** da **hora** de cada máquina.

💡 Puxa para o kit: **TAREFA 2** — busca por similaridade no serviço de recuperação. Repositório: `sd-2026-2-kit-c2a2`.

Atividade para casa

1. **Amplie a simulação** para **4 processos** e pelo menos **3 mensagens** trocadas.
2. **Explique por escrito** (5 linhas): escolha **um par causal** e **um par concorrente** da sua execução e justifique **pelos vetores**.
3. **Avance o C2.A2**: ponha o **serviço de recuperação** (busca por similaridade) no ar.

📌 **Entregar até a próxima aula:** `lamport_sim.py` com 4 processos + a explicação escrita + o serviço de recuperação iniciado.

◆ Foco ENADE

O que costuma cair:

- Sincronização de relógios físicos e **desvio (clock drift)**.
- Relógio de **Lamport** e relação "aconteceu antes".
- **Relógios vetoriais** e detecção de **concorrência**.
- Ordenação de eventos e **exclusão mútua distribuída**.

Termos-chave: Relógio lógico · Lamport · Relógio vetorial · Causalidade · Eventos concorrentes · Clock drift

💡 Quase certo na prova: **calcular** um Lamport após um recebimento (máximo + 1) e **classificar** dois eventos (causal × concorrente) por vetores.

Questão de autoavaliação (estilo ENADE)

No processo **P1**, um evento leva seu relógio de Lamport a **3**. P1 envia uma mensagem a **P2** com o carimbo **3**. No recebimento, o relógio de **P2** valia **7**. Após o recebimento, o relógio de P2 passa a valer:

- A) 3
- B) 7
- C) **8**
- D) 10
- E) 4

Resolução — alternativa C

- Regra do **recebimento**: **máximo**(relógio local, carimbo) + 1.
- **máx(7, 3) + 1 = 8.**
- Cai em quem esquece o "+1" (marca 7) ou quem **soma** tudo (marca 10).

💡 É a linha **d** da nossa tabela, com outros números: $\text{máx}(\text{local}, \text{carimbo}) + 1$.

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 14 — Tempo e estados globais.
- O artigo clássico de **Lamport** (1978) — leitura opcional.
- Rode o `lamport_sim.py` mudando a **ordem** dos eventos e observe.

Glossário

- **Relógio lógico**: contador que ordena eventos sem depender da hora real.
- **Lamport**: um contador por processo (máximo + 1).
- **Relógio vetorial**: um vetor de contadores; detecta concorrência.
- **Causalidade**: um evento influenciou o outro.
- **Concorrentes**: sem relação de causa entre si.
- **Clock drift**: desvio entre relógios físicos.

Até a próxima aula

Amplie a simulação (4 processos) e avance o serviço de recuperação.

Próxima (Aula 10): Replicação, consistência e o teorema CAP — quando duas cópias divergem, o que a concorrência de hoje ajuda a detectar.

← Anterior

Aula 8 · Mensageria/fila

≡ Índice

todas as aulas